

UserSpace

Clean Architecture + Dependency Injection
Learning Guide

A step-by-step learning document for building the Users feature from first principles.

Learning philosophy

Do not memorize folders or annotations. Start from a problem, identify responsibility, choose the boundary, implement one class, and then understand how dependencies are connected.

Project: UserSpace

Feature: Users

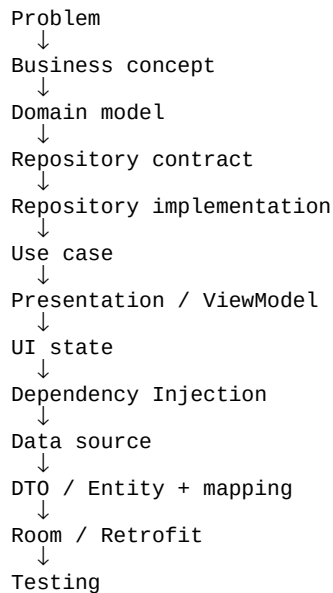
Focus: Clean Architecture, Hilt / DI, StateFlow, data modelling, Room, Retrofit and testing

1. The learning process

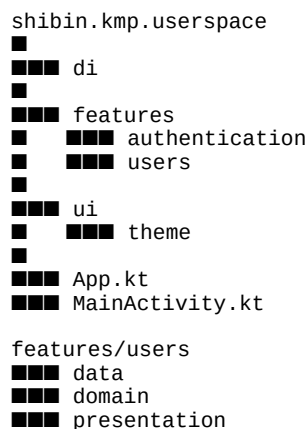
Rule: We build UserSpace one concept at a time. Each step follows: **Problem** → **Concept** → **Responsibility** → **Code** → **Review** → **Next dependency**.

The goal is to be able to look at an Android project and explain why a class exists, which layer owns it, who creates it, what it depends on, and how it can be replaced or tested.

The overall mental model



2. UserSpace base structure



We intentionally keep the top-level structure small. Each feature owns its own Clean Architecture layers. We avoid creating large global data/domain/presentation packages for every feature.

3. Step 1 — Domain Model

Problem

UserSpace needs to represent a User. Before thinking about APIs, databases or UI, define the application's business concept.

Concept

The Domain layer contains business concepts and rules. A domain model should not care whether data comes from Room, Retrofit, Firebase or hardcoded data.

```
features/users/domain/model/User.kt

data class User(
    val id: Int,
    val name: String,
    val email: String,
    val city: String
)
```

Key lesson

The domain User is the application's representation of a user. External technologies should not define this model.

4. Step 2 — Repository Contract

Problem

The application needs users, but the Domain should not know where they come from.

Concept

A repository is an abstraction for obtaining or changing domain data. The Domain owns the repository interface; the Data layer owns its implementation.

```
features/users/domain/repository/UserRepository.kt

interface UserRepository {

    suspend fun getUsers(): List<User>
}
```

Dependency direction

```
Domain
    UserRepository ← contract

Data
    UserRepositoryImpl → implementation
```

This is the beginning of Dependency Inversion: high-level business code depends on an abstraction rather than a concrete data source.

5. Step 3 — Repository Implementation

Problem

An interface describes what is needed, but something must actually provide the data.

```
features/users/data/repository/UserRepositoryImpl.kt

class UserRepositoryImpl : UserRepository {

    override suspend fun getUsers(): List<User> {
        return listOf(
            User(1, "John Doe", "john@gmail.com", "Chennai"),
            User(2, "Sarah Smith", "sarah@gmail.com", "Bangalore"),
            User(3, "Alex Johnson", "alex@gmail.com", "Mumbai")
        )
    }
}
```

For learning, hardcoded data is acceptable at this stage. The purpose is to understand the boundary before introducing Room or Retrofit.

6. Step 4 — Use Case

Problem

The application performs actions such as Get Users, Get User, Create User, Update User and Delete User.

Concept

A Use Case represents an application/business action. The ViewModel asks the Use Case to perform an action instead of directly coordinating repositories.

```
features/users/domain/usecase/GetUsersUseCase.kt

class GetUsersUseCase(
    private val userRepository: UserRepository
) {

    suspend operator fun invoke(): List<User> {
        return userRepository.getUsers()
    }
}
```

What does invoke mean?

suspend allows the function to call suspending operations such as API or database calls. **operator** allows the object to be called like a function. **invoke()** is therefore callable as `getUsersUseCase()`.

Using `invoke` is a Kotlin convenience, not a Clean Architecture requirement. `execute()` would also be valid.

7. Step 5 — ViewModel

Problem

The UI needs someone to trigger the use case and hold screen state.

Concept

The ViewModel coordinates UI actions with Use Cases and exposes UI state. It should not know about RepositoryImpl, Retrofit or Room.

```
@HiltViewModel
class UserViewModel @Inject constructor(
    private val getUsersUseCase: GetUsersUseCase
) : ViewModel() {

    // UI state and actions live here.
}

Compose UI
  ↓
UserViewModel
  ↓
GetUsersUseCase
  ↓
UserRepository
  ↓
UserRepositoryImpl
```

8. Step 6 — UI State

Problem

A screen can be loading, successful, or in an error state. These should be represented explicitly.

```
sealed interface UserListUiState {

    data object Loading : UserListUiState
    data class Success(
```

```

        val users: List<User>
    ) : UserListUiState

    data class Error(
        val message: String
    ) : UserListUiState
}

```

UI state belongs to Presentation because Loading/Success/Error describe the state of this screen, not a business entity.

9. Step 7 — StateFlow

```

private val _uiState =
    MutableStateFlow<UserListUiState>(
        UserListUiState.Loading
    )

val uiState: StateFlow<UserListUiState> =
    _uiState.asStateFlow()

```

The MutableStateFlow stays private so only the ViewModel can change it. The UI receives a read-only StateFlow.

```

fun loadUsers() {
    viewModelScope.launch {
        _uiState.value = UserListUiState.Loading

        try {
            val users = getUsersUseCase()
            _uiState.value = UserListUiState.Success(users)
        } catch (e: Exception) {
            _uiState.value = UserListUiState.Error(
                e.message ?: "Something went wrong"
            )
        }
    }
}

```

10. Step 8 — Compose observes state

```

@Composable
fun UserListScreen(
    viewModel: UserViewModel = hiltViewModel()
) {
    val uiState by viewModel.uiState.collectAsState()

    LaunchedEffect(Unit) {
        viewModel.loadUsers()
    }

    when (val state = uiState) {
        UserListUiState.Loading -> {
            CircularProgressIndicator()
        }

        is UserListUiState.Success -> {
            state.users.forEach { user ->
                Text("${user.name} - ${user.email}")
            }
        }

        is UserListUiState.Error -> {
            Text(state.message)
        }
    }
}

```

Compose observes StateFlow. When the ViewModel updates the state, Compose recomposes and displays the appropriate UI.

11. Step 9 — Dependency Injection with Hilt

Why DI?

Without DI, object construction becomes manual and spreads through the application.

```
val dataSource = UserDataSource()
val repository = UserRepositoryImpl(dataSource)
val useCase = GetUsersUseCase(repository)
val viewModel = UserViewModel(useCase)
```

Dependency Injection moves responsibility for constructing this graph to a container such as Hilt.

```
UserViewModel
  ↓
GetUsersUseCase
  ↓
UserRepository
  ↓
UserRepositoryImpl
  ↓
UserDataSource
```

@Inject constructor

```
class GetUsersUseCase @Inject constructor(
    private val userRepository: UserRepository
) {
    suspend operator fun invoke(): List<User> {
        return userRepository.getUsers()
    }
}
```

@Inject constructor tells Hilt that it can construct this class using that constructor.

@Binds

```
@Module
@InstallIn(SingletonComponent::class)
abstract class UserRepositoryModule {

    @Binds
    abstract fun bindUserRepository(
        implementation: UserRepositoryImpl
    ): UserRepository
}
```

@Binds tells Hilt which implementation to use when an interface is requested: UserRepository → UserRepositoryImpl.

@InstallIn

@InstallIn(SingletonComponent::class) makes the module's bindings available in the application's Hilt component. @InstallIn is about where bindings live in the Hilt graph; it is not the same thing as @Singleton.

12. Step 10 — DataSource

Problem

The repository should not also be responsible for knowing the details of how data is physically obtained.

```
class UserDataSource @Inject constructor() {
    suspend fun getUsers(): List<User> {
        return listOf(
            User(1, "John Doe", "john@gmail.com", "Chennai"),
            User(2, "Sarah Smith", "sarah@gmail.com", "Bangalore"),
            User(3, "Alex Johnson", "alex@gmail.com", "Mumbai")
        )
    }
}

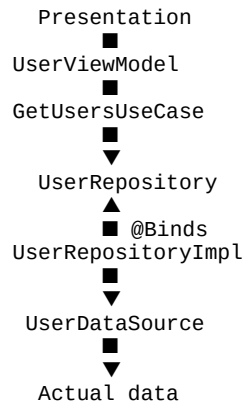
class UserRepositoryImpl @Inject constructor(
    private val userDataSource: UserDataSource
) : UserRepository {

    override suspend fun getUsers(): List<User> {
        return userDataSource.getUsers()
    }
}
```

```
}
}
```

Later, this DataSource will become a LocalDataSource backed by Room or a RemoteDataSource backed by Retrofit. The Repository will coordinate those sources.

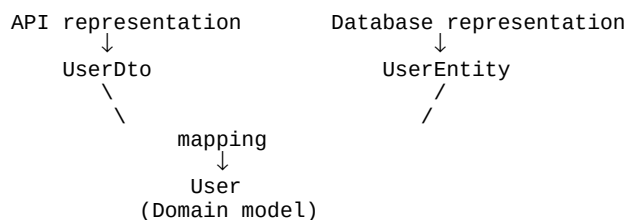
13. Current dependency graph



The key idea is that Domain defines the contract, Data implements the contract, Presentation uses the Domain use case, and Hilt constructs the dependency graph.

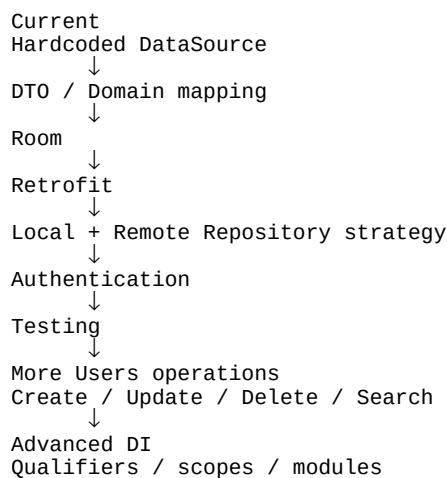
14. Next learning stage — Data models

The current DataSource returns the Domain User directly. That is intentionally simplified for learning. The next architectural improvement is to separate external data representations from the Domain model.



This separation protects the Domain from API and database changes.

15. Planned progression



Do not implement everything at once. Each stage should be understood and reviewed before moving forward.

16. Questions to ask yourself

- Why does User belong to Domain?
- Why is UserRepository an interface?
- Why does UserRepositoryImpl belong to Data?
- Why does a Use Case depend on the repository interface?
- Why should the ViewModel not call Retrofit directly?
- What problem does Dependency Injection solve?
- What does @Inject constructor actually tell Hilt?
- Why do we need @Binds for an interface?
- What does @InstallIn control?
- Why should UI state stay in Presentation?
- Why should DTOs and Room entities be separate from Domain models?
- If the API changes its JSON fields, which classes should change?

17. UserSpace learning rule

Never add an architecture component just because a tutorial says to. First identify the problem. Then choose the responsibility. Then introduce the smallest abstraction that solves it.

The purpose of UserSpace is not to create a perfect sample app. It is to train architectural thinking.

18. GitHub usage

This PDF can live in the repository as a learning companion, for example:

```
docs/  
■■■ UserSpace-Clean-Architecture-DI-Learning-Guide.pdf
```

Keep the source code as the practical exercise and this document as the explanation of the decisions behind the code.